
RAPPORT DE GÉNIE LOGICIEL ET CONDUITE DE PROJET

Application des concepts de Génie Logiciel
et de Conduite de Projet au développement de
SaaS Directory

Présenté par :

Omar SARSAR

Iliass HARIZ

Soulaymane ELFAKIR

2026

Dédicace

*Nous dédions ce travail à nos familles pour leur soutien indéfectible,
à nos enseignants pour leur guidance et leurs enseignements,
et à tous ceux qui croient en l'innovation technologique au service de la
communauté.*

Remerciements

Nous tenons à exprimer notre profonde gratitude envers l'ensemble de nos enseignants qui nous ont accompagnés tout au long de notre parcours académique. Leurs conseils avisés, leur patience et leur exigence ont été des facteurs déterminants dans la réussite de ce projet.

Nous remercions également nos camarades de promotion pour les échanges enrichissants et l'entraide collective qui ont contribué à notre progression.

Ce rapport présente l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de **SaaS Directory**, une plateforme de marketplace dédiée aux produits logiciels en mode SaaS. Le projet a été réalisé avec une stack technologique moderne comprenant Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL et Better Auth 1.6.9.

L'étude couvre l'ensemble des neuf chapitres du cours de Génie Logiciel : rappels fondamentaux, généralités, cycle de vie du logiciel, démarche de conception, normalisation, tests, gestion de projet, analyse des risques et conduite du changement. Chaque concept théorique est illustré par sa mise en oeuvre concrète dans le projet.

Le rapport inclut une estimation du coût par la méthode COCOMO appliquée à la base de code réelle de 12 166 lignes, une analyse des risques liés aux breaking changes de Next.js 16.2.4, ainsi qu'une présentation détaillée de la gestion des versions via Git et des migrations Drizzle ORM.

Mots-clés : Génie Logiciel, Conduite de Projet, Next.js, React, Drizzle ORM, PostgreSQL, CO-COMO, Cycle de Vie, UML, Normalisation, Tests Logiciels, Gestion des Risques.

This report presents the systematic application of Software Engineering and Project Management concepts to the development of **SaaS Directory**, a marketplace platform dedicated to SaaS software products. The project was built using a modern technology stack including Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL, and Better Auth 1.6.9.

The study covers all nine chapters of the Software Engineering course : fundamental recalls, general concepts, software life cycle, design methodology, standardization, testing, project management, risk analysis, and change management. Each theoretical concept is illustrated by its concrete implementation in the project.

The report includes a COCOMO cost estimation applied to the actual codebase of 12,166 lines, a risk analysis related to Next.js 16.2.4 breaking changes, and a detailed presentation of version control via Git and Drizzle ORM migrations.

Keywords : Software Engineering,Project Management,Next.js, React,Drizzle ORM,PostgreSQL, COCOMO,Life Cycle,UML,Standardization,Software Testing,Risk Management.

Dédicace	2
Remerciements	2
Résumé	3
Abstract	4
Introduction Générale	10
1 Chapitre 1 : Rappels	11
1.1 Définitions Fondamentales	11
1.1.1 Logiciel et Application	11
1.1.2 Système d'Information (SI)	11
1.1.3 Méthode, Outil et Méthodologie	11
1.2 Application à SaaS Directory	12
2 Chapitre 2 : Génie Logiciel — Généralités	12
2.1 Objectifs du Génie Logiciel	12
2.2 Les Principes du GL Appliqués	12
2.2.1 Rigueur	12
2.2.2 Séparation des Problèmes	12
2.2.3 Modularité	12
2.2.4 Abstraction	12
2.2.5 Généricity	14
2.2.6 Construction Incrémentale	15
2.2.7 Anticipation du Changement	15
2.3 Les Attributs du Logiciel	15
2.3.1 Couplage	15
2.3.2 Complexité	15
2.3.3 Extensibilité	15

2.3.4	Lisibilité et Visibilité du Comportement	17
3	Chapitre 3 : Représentations du Cycle de Vie du Logiciel	17
3.1	Les Modèles de Cycle de Vie	17
3.2	Choix et Justification du Modèle	17
3.3	Application au Projet SaaS Directory	17
4	Chapitre 4 : La Démarche	18
4.1	Méthodes de Développement Semi-Formelles	18
4.2	Diagrammes UML Appliqués au Projet	18
4.2.1	Diagramme de Cas d'Utilisation	18
4.2.2	Diagramme de Classes	19
4.2.3	Diagramme de Séquence	22
4.3	Méthodes Formelles et Langage Z	22
5	Chapitre 5 : La Normalisation	23
5.1	La Norme ISO et la Certification ISO 9000	23
5.2	Les Normes IEEE	23
5.3	Normalisation Appliquée au Projet	23
5.3.1	Normes de Codage	23
5.3.2	Architecture Normalisée	24
5.3.3	Gestion Documentaire et Traçabilité	24
6	Chapitre 6 : Les Tests	24
6.1	Types de Tests et Stratégie	24
6.1.1	Tests Unitaires	24
6.1.2	Tests End-to-End	24
6.1.3	Pyramide des Tests	24
6.1.4	Tests de Performance	25
6.2	Outils et Environnement de Test	26
6.3	Couverture et Résultats	26

7	Chapitre 7 : La Gestion des Projets Logiciels	26
7.1	Organisation du Projet (WBS, PBS, OBS)	27
7.2	Planification (GANTT, PERT)	27
7.3	Estimation des Coûts par COCOMO	27
7.3.1	Taille du Projet	28
7.3.2	Application du COCOMO de Base	28
8	Chapitre 8 : L'Analyse des Risques	28
8.1	Identification des Risques	28
8.1.1	Risques Techniques	28
8.1.2	Risques de Sécurité	29
8.1.3	Risques de Performance	29
8.2	Analyse et Stratégies de Mitigation	29
8.2.1	Mitigation des Risques Techniques	29
8.2.2	Mitigation des Risques de Sécurité	29
8.2.3	Mitigation des Risques de Performance	29
8.3	Plan de Continuité	30
9	Chapitre 9 : Conduite du Changement	30
9.1	Nature des Changements	30
9.2	Gestion des Versions	30
9.3	Adaptation et Communication	30
	Conclusion Générale et Perspectives	31
	Bibliographie et Webographie	32
	Annexes	33
	Annexe A : Dependances Principales	33
	Annexe B : Diagramme de Deploiement	34

1	Architecture en Couches — SaaS Directory	13
2	Diagramme d'Activité — Flux d'Authentification	14
3	Diagramme d'État — Cycle de Vie des Abonnements	16
4	Diagramme de Cas d'Utilisation — SaaS Directory	19
5	Diagramme de Classes — Package Authentification (4 tables)	20
6	Diagramme de Classes — Package Annuaire (Directory) (8 tables)	21
7	Diagramme de Classes — Package Social & Communication (8 tables)	21
8	Diagramme de Séquence — Envoi d'un Message	22
9	Pyramide des Tests — Stratégie de validation de SaaS Directory	25
10	Impact du Cache Redis sur les Performances — Comparaison avant/après	26
11	Diagramme de GANTT — Planification du Projet SaaS Directory	27
12	Chronologie de l'Évolution du Schéma — Migrations Drizzle	31
13	Diagramme de Deploiement — Architecture Physique de SaaS Directory	34

1	Résultats du benchmark Cache Redis — SaaS Directory (10 mai 2026)	25
2	Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)	28
3	Dependances principales du projet SaaS Directory	33

Dans un monde de plus en plus numérisé, le développement de logiciels est devenu une discipline complexe qui exige rigueur, méthodologie et gestion efficace des ressources. Le Génie Logiciel (GL) et la Conduite de Projet fournissent le cadre théorique et pratique nécessaire pour mener à bien des projets logiciels de qualité, dans les délais et budgets impartis.

Ce rapport a pour objectif de démontrer l'application concrète des concepts théoriques du Génie Logiciel et de la Conduite de Projet au développement de **SaaS Directory**, une application web moderne permettant aux créateurs de logiciels de lancer, promouvoir et vendre leurs produits SaaS.

SaaS Directory est une plateforme de marketplace dédiée aux produits logiciels en mode SaaS (Software as a Service). L'application offre un catalogue où les utilisateurs peuvent découvrir des outils logiciels, voter pour leurs produits préférés, interagir via des commentaires et une messagerie, et souscrire à des plans d'abonnement Free ou Pro.

Le projet a été réalisé avec une stack technologique contemporaine et cohérente : Next.js 16.2.4 (framework React avec App Router), React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2 (ORM type-safe pour PostgreSQL), Better Auth 1.6.9 (authentification complète avec OAuth et 2FA), Redis (cache côté serveur), et Playwright 1.59.1 pour les tests end-to-end.

Le codebase compte exactement **20 tables** dans le schéma Drizzle ORM, **12 166 lignes de code** réparties sur **108 fichiers** TypeScript/TSX, avec **4 migrations** versionnées (0000 à 0003). Le projet est versionné sur Git avec une branche principale et des branches de fonctionnalités.

Ce rapport s'articule autour de neuf chapitres principaux. Le premier chapitre présente les rappels fondamentaux sur les concepts de base. Le deuxième traite des généralités du Génie Logiciel et de ses principes. Le troisième décrit les modèles de cycle de vie. Le quatrième expose la démarche de conception avec UML. Le cinquième aborde la normalisation. Le sixième détaille les stratégies de tests. Le septième présente la gestion de projet et l'estimation COCOMO. Le huitième analyse les risques. Le neuvième et dernier chapitre traite de la conduite du changement.

1.1 Définitions Fondamentales

Avant d'aborder les concepts avancés du Génie Logiciel, il est essentiel de poser les bases terminologiques et conceptuelles.

1.1.1 Logiciel et Application

Un **logiciel** est défini comme un ensemble de programmes, procédures, règles et documentations relatifs au fonctionnement d'un ensemble de traitements d'informations. Une **application** est un logiciel qui sert à implanter un système d'informations. On distingue trois grandes catégories : applications de gestion (peu de traitement, beaucoup de données), applications scientifiques (beaucoup de traitement, peu de données), et applications industrielles (peu de traitement et de données).

SaaS Directory s'inscrit dans la catégorie des applications de gestion/web, avec une composante significative de données structurées (20 tables relationnelles).

1.1.2 Système d'Information (SI)

Le **Système d'Information (SI)** constitue l'ensemble des flux d'opérations, d'informations et de moyens mis en oeuvre par une organisation. Le **Système d'Information Automatisé (SIA)** repose sur la technologie informatique. SaaS Directory est un SIA complet : il traite des profils utilisateurs, des produits, des votes, des messages et des abonnements via une infrastructure web moderne.

1.1.3 Méthode, Outil et Méthodologie

Une méthode est un processus discipliné qui génère un ensemble de modèles décrivant les différents aspects d'un système logiciel, en utilisant une notation bien définie.

Un **outil** désigne un programme ou un ensemble de programmes aidant à la mise en oeuvre des techniques. L'**Atelier de Génie Logiciel (AGL)** regroupe l'ensemble des outils utilisés pour le développement, la maintenance et la documentation.

La **méthodologie** est la démarche générale qui fait appel à des méthodes et des outils pour aboutir au logiciel final. Pour SaaS Directory, la méthodologie adoptée combine l'approche incrémentale itérative avec les principes du cycle de vie en V.

1.2 Application à SaaS Directory

SaaS Directory s’inscrit dans le cadre théorique des Systèmes d’Information Automatisés. En tant que marketplace web, il traite de grands volumes de données structurées et exige une architecture logicielle rigoureuse pour garantir performance, sécurité et évolutivité.

2.1 Objectifs du Génie Logiciel

Le **Génie Logiciel (GL)** vise à utiliser l’informatique dans les entreprises comme outil quotidien, à réduire les coûts de développement et de maintenance, à améliorer la qualité des logiciels, et à augmenter la productivité des équipes. Ces objectifs s’inscrivent dans une démarche globale de maîtrise du cycle de vie du logiciel.

2.2 Les Principes du GL Appliqués

2.2.1 Rigueur

La rigueur se traduit par l’utilisation de TypeScript (typage statique strict), les conventions de nommage systématiques (PascalCase pour les composants, camelCase pour les fonctions), et la revue de code via les pull requests Git. La documentation interne du projet documente les conventions spécifiques de développement.

2.2.2 Séparation des Problèmes

Trois niveaux de séparation sont appliqués : séparation dans le temps (développement par phases avec tests à chaque niveau), séparation dans l’espace (architecture en couches, voir figure 1), et séparation en sous-systèmes (modules application, composants et utilitaires serveur).

2.2.3 Modularité

L’architecture repose sur des composants React indépendants, chacun responsable d’une fonctionnalité spécifique. Le schéma Drizzle ORM décompose le système en 20 tables distinctes, chacune encapsulant une entité métier. Cette modularité facilite la maintenance et les tests unitaires.

2.2.4 Abstraction

Plusieurs niveaux d’abstraction sont utilisés : les Server Actions abstraient la logique métier du rendu UI ; Drizzle ORM abstrait les requêtes SQL complexes ; le système de cache Redis abstrait

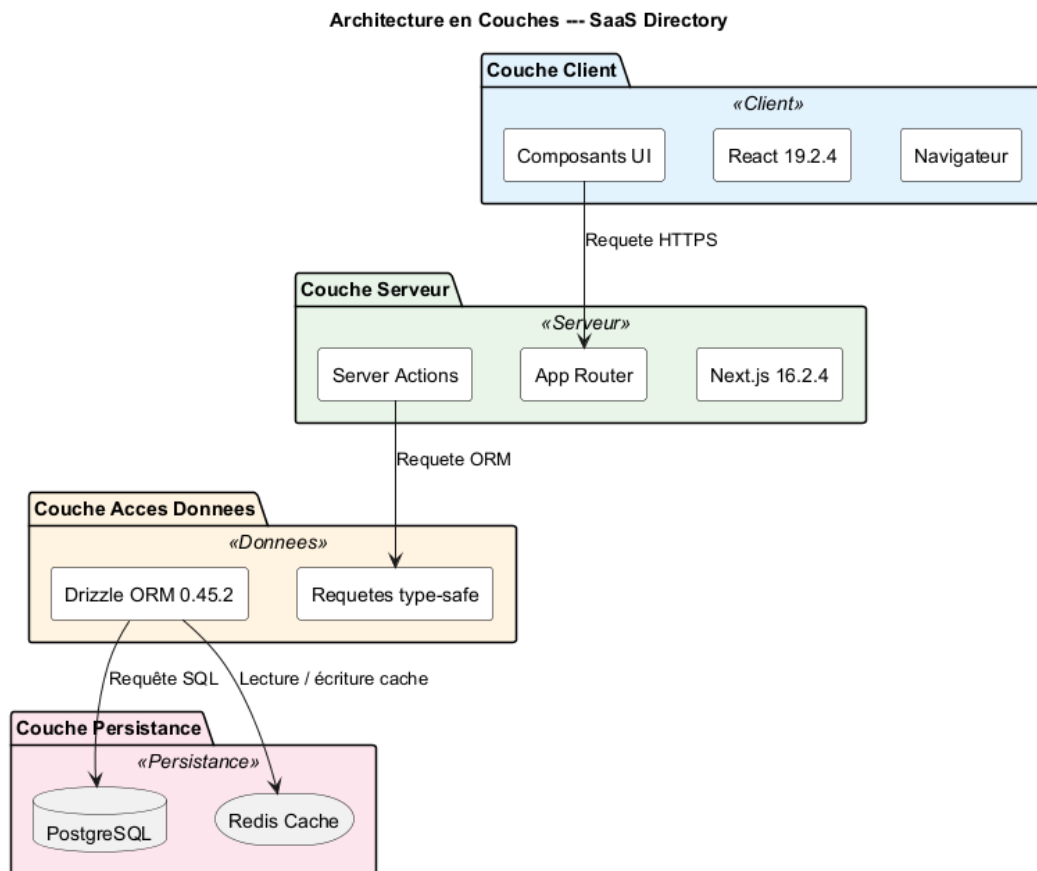


FIGURE 1 – Architecture en Couches — SaaS Directory

la gestion des données temporaires.

2.2.5 Généricity

Les composants UI sont conçus comme des composants réutilisables paramétrables via les propriétés React. Le système d'authentification Better Auth est générique et supporte OAuth (Google, GitHub) et l'authentification par email/mot de passe. Le diagramme d'activité de la figure 2 illustre le flux complet d'authentification.

Diagramme d'Activite --- Flux d'Authentification

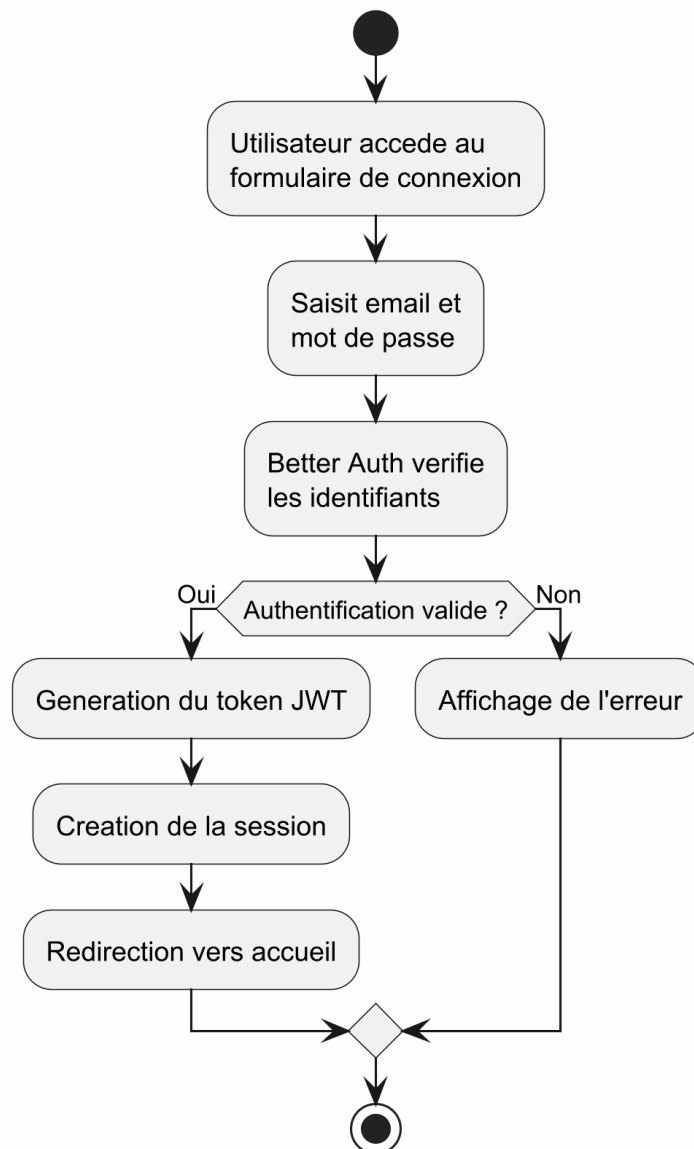


FIGURE 2 – Diagramme d'Activité — Flux d'Authentification

2.2.6 Construction Incrémentale

Le développement a suivi une approche incrémentale documentée dans l'historique Git. Les phases principales sont : fondations (Next.js 16.2.4, shadcn, Drizzle), infrastructure de tests (Vitest, Playwright), base de données (schéma Drizzle, migrations, seed), authentification TDD (login/signup, tests), fonctionnalités core (marketplace, profils, messages, notifications), et polissage final (recherche, responsive, build fixes).

2.2.7 Anticipation du Changement

L'architecture anticipe plusieurs évolutions : le schéma Drizzle inclut déjà les champs client et abonnement Stripe pour une future intégration du paiement ; le système de plans (Free/Pro) permet d'ajouter de nouveaux niveaux d'abonnement. Le diagramme de la figure 3 présente le cycle de vie des abonnements.

2.3 Les Attributs du Logiciel

L'analyse du codebase de 12 166 lignes réparties sur 108 fichiers permet d'évaluer ces attributs.

2.3.1 Couplage

Le couplage est faible grâce à l'architecture par composants React et l'encapsulation des Server Actions. Chaque table Drizzle ORM est indépendante, avec des relations explicites via les clés étrangères. Le couplage entre frontend (React) et backend (Server Actions) est réduit au minimum par l'API interne de Next.js 16.2.4.

2.3.2 Complexité

Avec 20 tables relationnelles et 12 166 lignes de code, le système présente une complexité modérée. Cette complexité est maîtrisée par la modularité, le typage statique TypeScript, et la documentation inline (JSDoc).

2.3.3 Extensibilité

L'extensibilité est démontrée par la facilité d'ajouter de nouvelles tables au schéma Drizzle (4 migrations versionnées), l'ajout de nouveaux composants UI sans impacter les existants, et la configuration modulaire du framework.

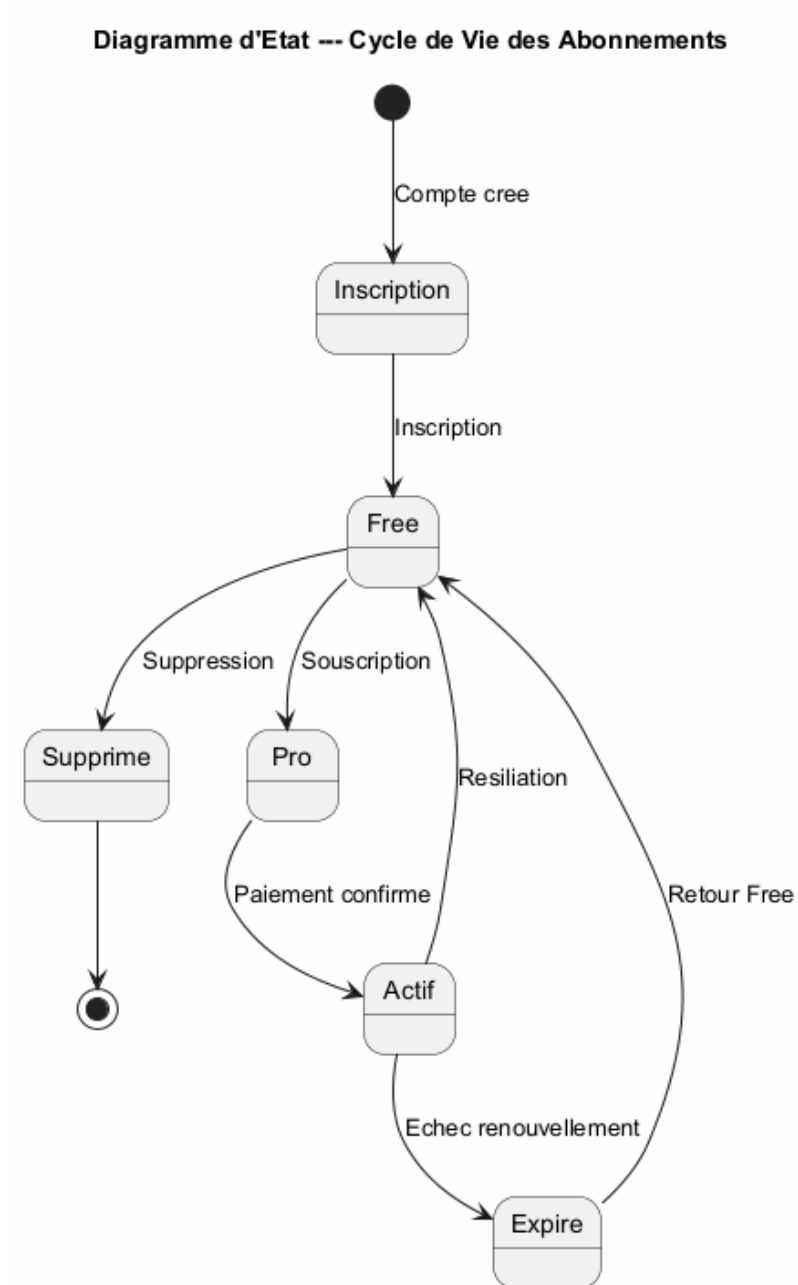


FIGURE 3 – Diagramme d'État — Cycle de Vie des Abonnements

2.3.4 Lisibilité et Visibilité du Comportement

La lisibilité est assurée par TypeScript (typage explicite), les conventions de nommage strictes, et la documentation JSDoc. La visibilité du comportement est garantie par les tests unitaires (10 fichiers de test couvrant l'authentification, la souscription, le schéma, les actions serveur) et les tests end-to-end (Playwright 1.59.1).

Le développement d'un logiciel n'est pas simplement l'écriture d'un programme. La taille et la complexité des logiciels actuels exigent l'utilisation de processus de développement structurés, appelés **cycles de vie du logiciel**.

3.1 Les Modèles de Cycle de Vie

Le modèle en cascade (waterfall) suit une séquence linéaire de phases : spécification, conception, codage, tests, et maintenance. Le modèle en V associe à chaque phase de développement une phase de test correspondante. Le modèle itératif et incrémental divise le projet en itérations courtes, chacune livrant une version fonctionnelle du logiciel.

3.2 Choix et Justification du Modèle

Pour SaaS Directory, nous avons adopté un **modèle hybride** combinant l'approche incrémentale itérative avec des éléments du modèle en V. Cette décision repose sur trois arguments : la nature évolutive du projet (marketplace web), les contraintes temporelles académiques, et l'importance de la validation systématique (tests à chaque niveau).

3.3 Application au Projet SaaS Directory

Le développement de SaaS Directory s'est déroulé sur environ trois semaines, du 5 au 21 mai 2026, comme l'atteste l'historique Git du projet. Le travail a été réparti entre trois développeurs selon une approche incrémentale itérative.

Phase 1 : Fondations (5 mai). Initialisation du projet Next.js 16.2.4 avec TypeScript et Tailwind CSS v4, configuration de Drizzle ORM 0.45.2 avec PostgreSQL, mise en place de Better Auth 1.6.9, et création des tables de base (user, session, account, verification).

Phase 2 : Infrastructure de tests (5–6 mai). Mise en place de Vitest 4.1.5, React Testing Library, Playwright 1.59.1, et configuration des seuils de couverture.

Phase 3 : Base de données et schéma (6 mai). Définition du schéma Drizzle ORM complet (20 tables) et génération progressive des 4 migrations versionnées (0000 à 0003), avec script de seed.

Phase 4 : Authentification et profils (6 mai). Développement des pages login/signup avec TDD, gestion des profils utilisateurs, et système de lancement de produits.

Phase 5 : Fonctionnalités core (6–7 mai). Développement du marketplace (votes, commentaires), messagerie en temps réel, et système d’abonnements Free/Pro.

Phase 6 : Polissage et optimisation (7–8 mai, 21 mai). Ajout de la recherche, reset de mot de passe, pagination infinie, corrections responsive, et build fixes.

La démarche de conception d’un logiciel définit la méthodologie suivie pour passer du besoin exprimé au logiciel opérationnel.

4.1 Méthodes de Développement Semi-Formelles

Le cours présente plusieurs méthodes semi-formelles. SADT (Structure Analysis and Design Technique) est une méthode structurée. MERISE est une méthode française systémique avec séparation données et traitements. UML/Processus Unifié est itératif, incrémental et orienté objet. OMT (Object Modelling Technique) a été proposé par James Rumbaugh. OOSE (Object-Oriented Software Engineering) a été développé par Ivar Jacobson.

Pour SaaS Directory, nous avons retenu l’approche UML/Processus Unifié pour plusieurs raisons : son caractère itératif et incrémental correspond à notre modèle de cycle de vie ; son orientation objet s’aligne avec notre stack technologique (React composants, Drizzle ORM entités) ; et UML est le standard industriel de facto pour la modélisation orientée objet.

4.2 Diagrammes UML Appliqués au Projet

4.2.1 Diagramme de Cas d’Utilisation

Le diagramme de cas d’utilisation modélise les interactions entre les acteurs et le système. Les acteurs principaux sont : Visiteur, Utilisateur Authentifié, Créateur de Produit, et Administrateur.

Diagramme de Cas d'Utilisation --- SaaS Directory

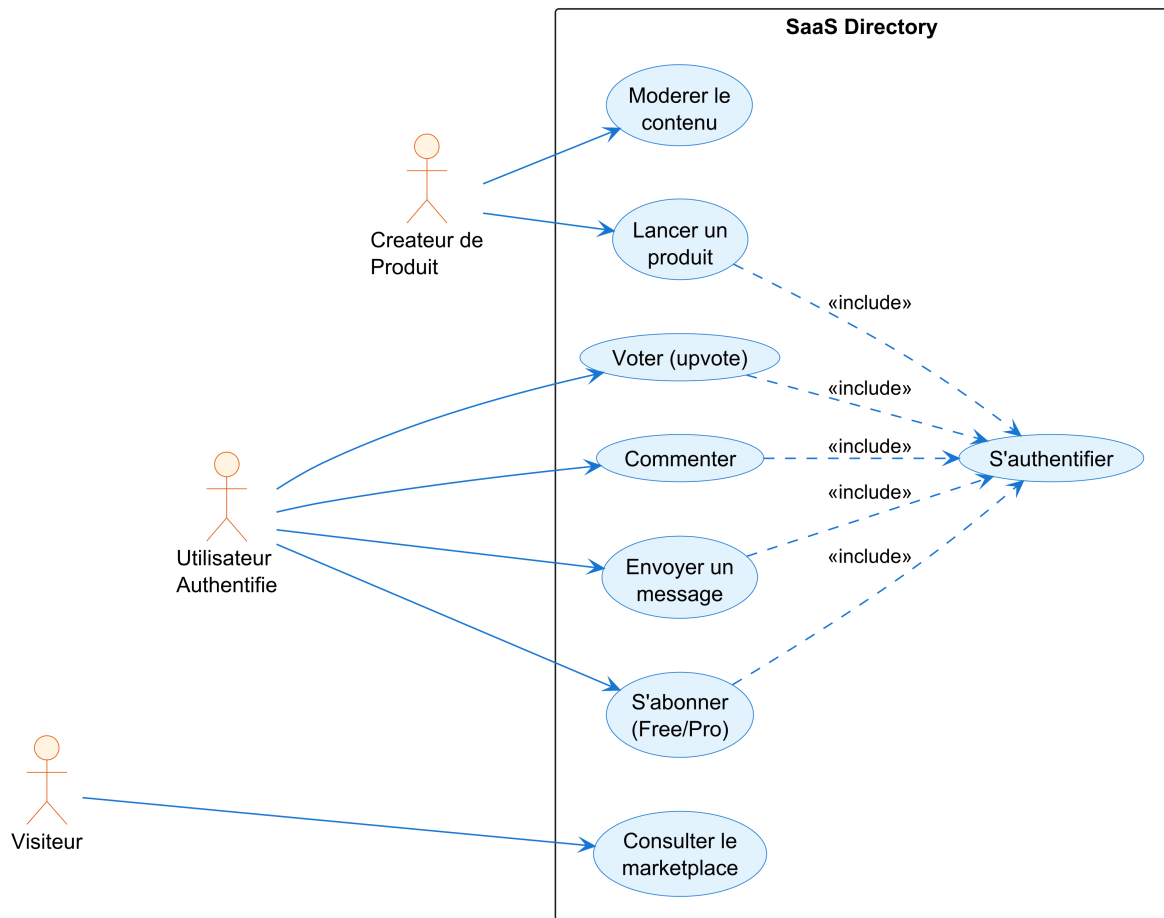


FIGURE 4 – Diagramme de Cas d'Utilisation — SaaS Directory

4.2.2 Diagramme de Classes

Le schéma Drizzle ORM constitue l'implémentation directe du diagramme de classes. Le système compte 20 tables regroupées en trois packages fonctionnels. Pour préserver la lisibilité, le diagramme est décomposé en trois sous-diagrammes. Les notes en bas de chaque diagramme indiquent les dépendances inter-packages sans introduire de classes externes fantômes.

Package Authentication. Ce package regroupe les quatre tables de gestion des identités : **User** (identifiant textuel, nom, email, vérification email, image), **Session** (token, expiration, adresse IP, user-agent), **Account** (comptes OAuth et locaux avec tokens, scopes, mots de passe), et **Vérification** (codes de vérification avec expiration). La classe User constitue le pivot du schéma avec des clés étrangères dans tous les autres packages.

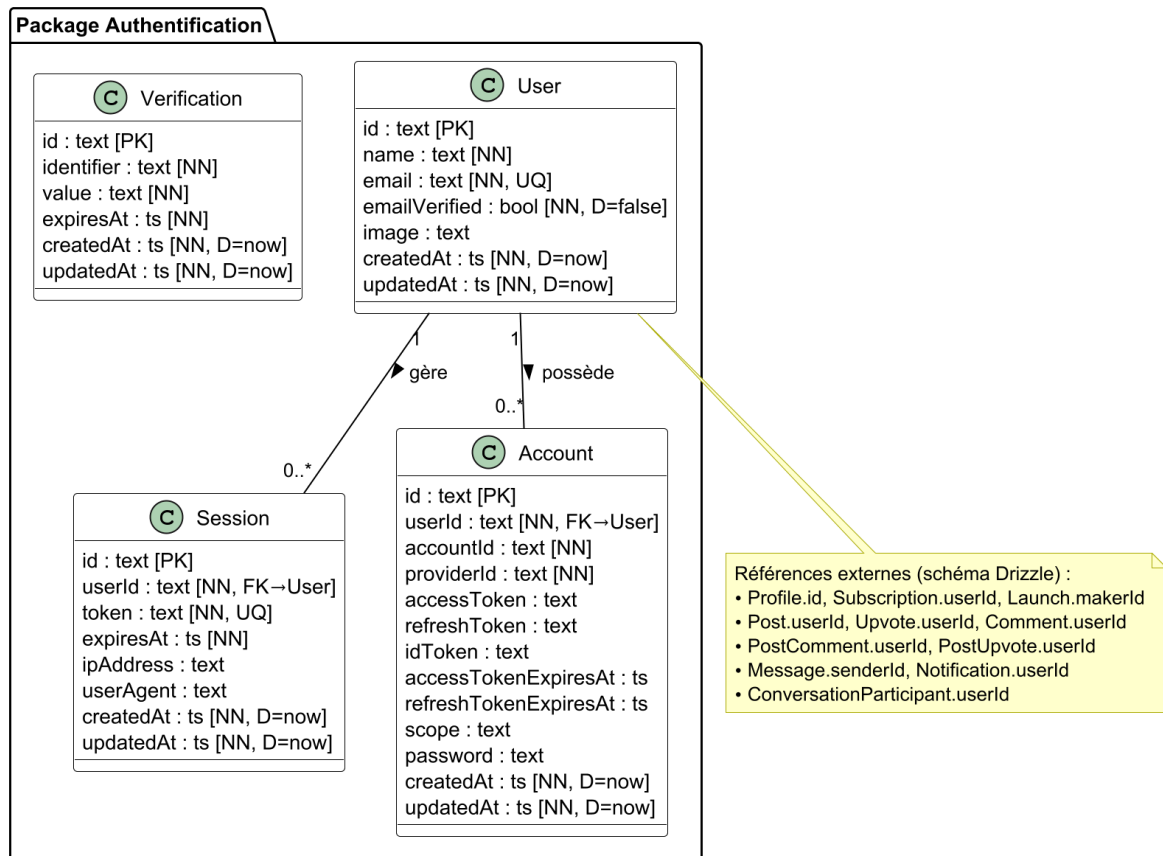


FIGURE 5 – Diagramme de Classes — Package Authentication (4 tables)

Package Annuaire (Directory). Ce package contient huit tables structurant le cœur métier du marketplace : **Profile** (profil utilisateur avec bio, liens sociaux, préférences JSONB), **Category** (catégories avec slug et couleur), **Plan** (abonnements Free/Pro avec identifiants Stripe et features JSON), **Subscription** (lien utilisateur-plan avec intervalle de facturation, statut, périodes Stripe), **Launch** (produit SaaS avec slug, tagline, URLs, prix, revenus récurrents, logo, votes et commentaires agrégés), **LaunchImage** (images associées avec ordre de tri), **LaunchCategory** (relation many-to-many composite Launch-Catégorie), et **Upvote** (vote utilisateur sur un produit, clé primaire composite). La table Launch est au centre du domaine avec des indexes sur maker, slug, à vendre et mis en avant.

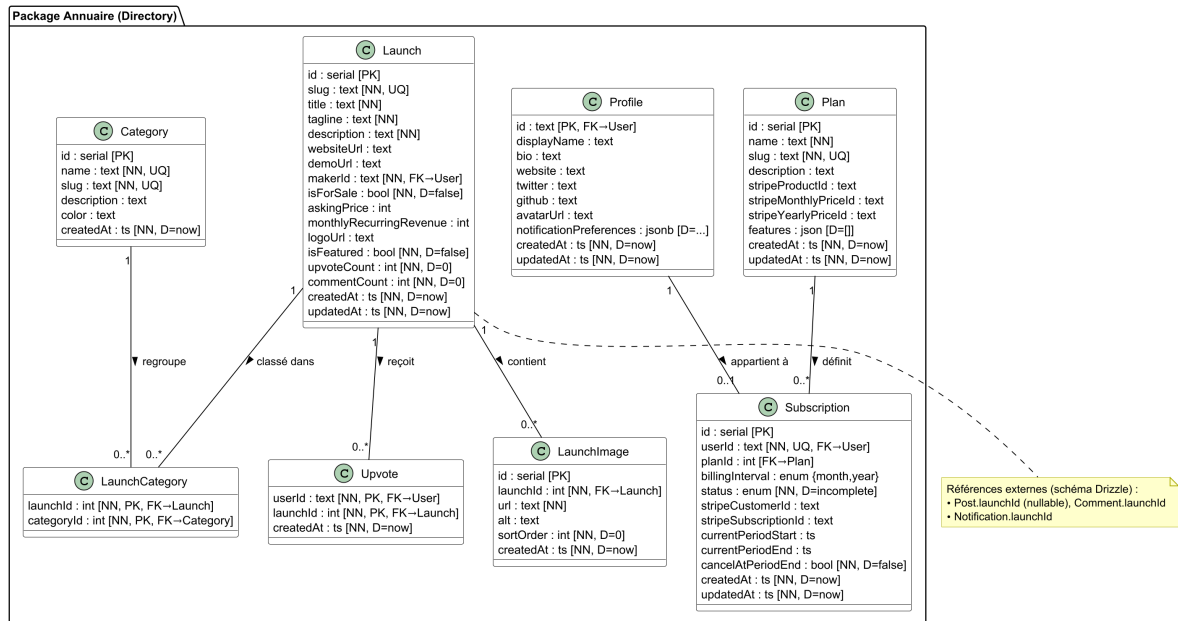


FIGURE 6 – Diagramme de Classes — Package Annuaire (Directory) (8 tables)

Package Social & Communication. Ce package rassemble huit tables gérant l'interaction utilisateur : **Post** (publications avec contenu, référence optionnelle vers un produit, compteurs de votes et commentaires), **PostUpvote** (vote sur une publication, clé primaire composite), **PostComment** (commentaire sur publication avec marqueur de suppression logique), **Comment** (commentaire sur un produit Launch), **Conversation** (canal de messagerie), **ConversationParticipant** (membre d'une conversation avec horodatage de dernière lecture, clé primaire composite), **Message** (message avec contenu, marqueur de suppression logique), et **Notification** (alerte typée avec références polymorphes vers acteur, produit, commentaire ou message).

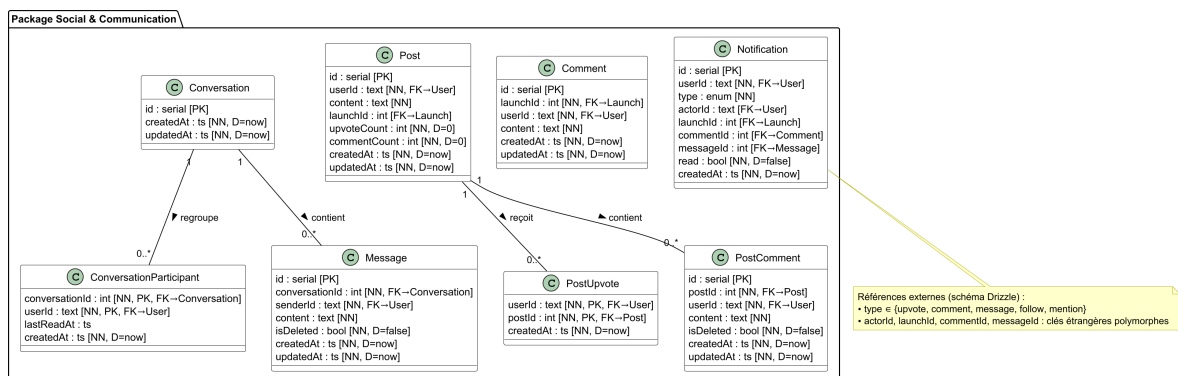


FIGURE 7 – Diagramme de Classes — Package Social & Communication (8 tables)

Cette décomposition modulaire reflète l'organisation fonctionnelle du schéma Drizzle ORM. Le package Authentification agit comme pivot central (toutes les tables référencent User), le package Annuaire structure le marketplace et la monétisation, et le package Social gère les interactions utilisateur (publications, messagerie, notifications).

4.2.3 Diagramme de Séquence

Le diagramme de séquence illustre l'interaction entre les acteurs et le système lors de l'envoi d'un message.

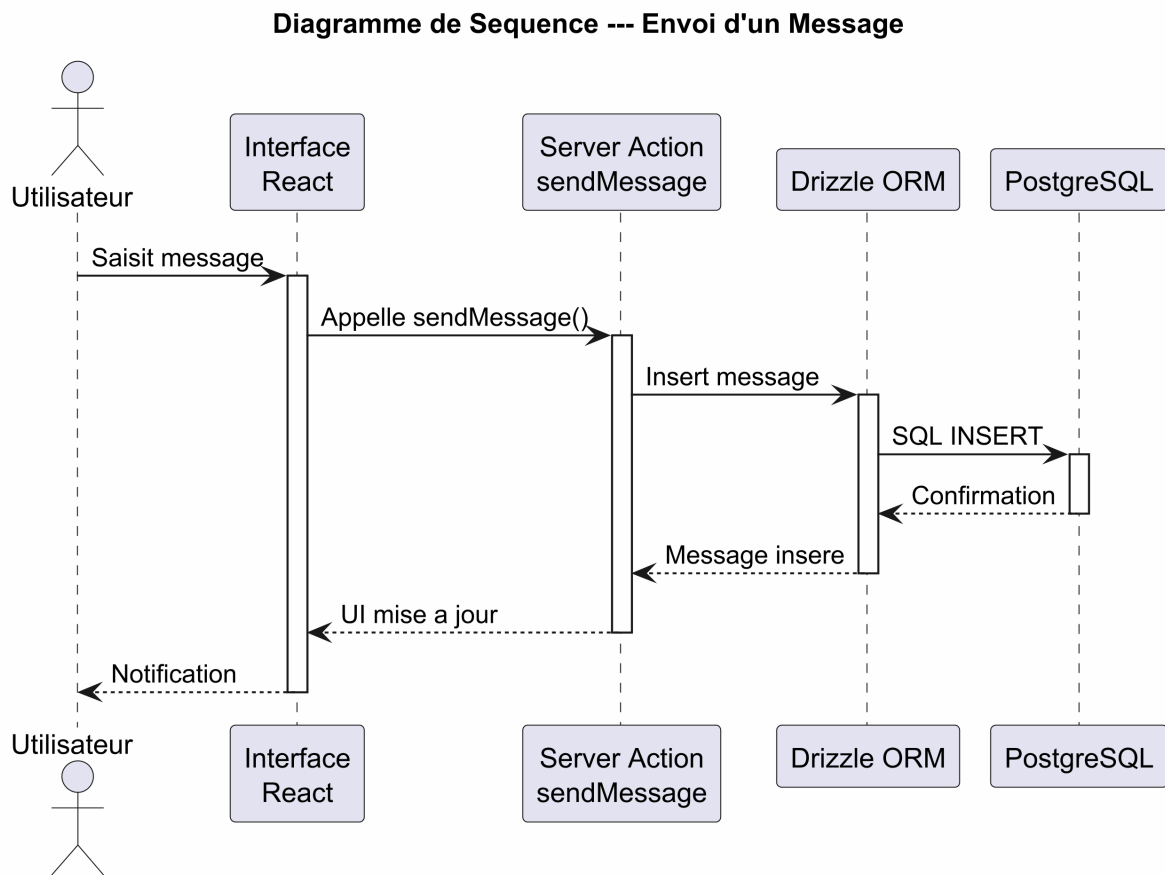


FIGURE 8 – Diagramme de Séquence — Envoi d'un Message

L'utilisateur émetteur saisit le message via l'interface React. Le composant client appelle l'action serveur qui insère le message dans la base de données via l'ORM. Cette architecture démontre la séparation des préoccupations entre présentation, métier et persistance.

4.3 Méthodes Formelles et Langage Z

Les méthodes formelles (langage Z, B, VDM) utilisent des notations mathématiques pour spécifier, développer et vérifier des systèmes logiciels. Le langage Z, développé par Jean-Raymond Abrial, est basé sur la théorie des ensembles et la logique des prédicats.

Ces méthodes n'ont pas été utilisées dans ce projet pour plusieurs raisons pragmatiques : le domaine du marketplace web n'est pas critique au sens des systèmes embarqués ; la courbe d'apprentissage est élevée ; et TypeScript avec Zod fournissent un niveau de vérification suffisant (typage statique, validation de schémas).

Les normes sont des apports documentés contenant des spécifications techniques destinées à être utilisées systématiquement comme règles ou lignes directrices.

5.1 La Norme ISO et la Certification ISO 9000

L'International Organization for Standardization (ISO), créée en 1947, est une fédération mondiale d'organismes nationaux de normalisation. La norme ISO 9000 traduit la démarche qualité au niveau de l'entreprise. Dans le contexte de SaaS Directory, la gestion documentaire est assurée par la documentation du projet, et la traçabilité est garantie par l'historique Git et les migrations Drizzle numérotées (0000 à 0003).

5.2 Les Normes IEEE

L'Institute of Electrical and Electronics Engineers (IEEE) a défini un ensemble de normes s'intéressant aux processus, produits et techniques de base du génie logiciel.

Fixe la signification des termes et choisit les processus prioritaires du cycle de vie du logiciel. Peut être considérée comme un dictionnaire et une liste de vérification pour la gestion des projets.

Recommandée pour les spécifications des exigences du logiciel. Pourrait être intégrée avec les approches de validation par prototype.

Pour appliquer la validation et la vérification. Norme facilement adaptable à tous les problèmes.

5.3 Normalisation Appliquée au Projet

5.3.1 Normes de Codage

Le projet respecte des conventions strictes : PascalCase pour les composants React, camelCase pour les fonctions, et kebab-case pour les routes API. Les imports sont organisés par groupe : React/Next.js, bibliothèques tierces, composants internes, utilitaires.

5.3.2 Architecture Normalisée

L'architecture suit le pattern MVC adapté à Next.js 16.2.4. Le **Modèle** est constitué par le schéma Drizzle ORM. La **Vue** est implémentée par les composants React. Le **Contrôleur** est assuré par les actions serveur du framework.

5.3.3 Gestion Documentaire et Traçabilité

La gestion documentaire est assurée par plusieurs artefacts : documentation de présentation et installation, conventions de développement, gestion des dépendances avec versions exactes, et messages de commit suivant une convention descriptive.

Les tests logiciels constituent une phase critique du cycle de vie du logiciel, visant à détecter les erreurs et à garantir que le système répond aux exigences spécifiées.

6.1 Types de Tests et Stratégie

La stratégie de tests de SaaS Directory repose sur une pyramide de tests classique.

6.1.1 Tests Unitaires

Les tests unitaires sont implémentés avec Vitest 4.1.5. Dix fichiers de test couvrent l'authentification, la souscription, le schéma de base de données, les actions serveur, et les composants UI.

6.1.2 Tests End-to-End

Les tests end-to-end sont réalisés avec Playwright 1.59.1. La configuration définit un projet Chromium avec tracing activé. Un fichier de test end-to-end valide le parcours utilisateur critique sur la page d'accueil.

6.1.3 Pyramide des Tests

La figure 9 illustre la stratégie de tests de SaaS Directory. Contrairement à la pyramide classique à trois niveaux, le projet ne compte pas de tests d'intégration distincts ; les 10 fichiers de test utilisent Vitest avec React Testing Library pour les tests unitaires et de composants, et un fichier Playwright couvre les parcours utilisateur complets.



FIGURE 9 – Pyramide des Tests — Stratégie de validation de SaaS Directory

6.1.4 Tests de Performance

La performance de SaaS Directory a été évaluée par benchmark comparatif mesurant l’impact du cache Redis sur les opérations de lecture fréquentes. Le test a été réalisé le 10 mai 2026 via le script `scripts/benchmark-cache.ts` qui compare les temps de réponse avec et sans cache distribué.

Architecture du cache. Le système utilise le pattern *Cache-Aside* (Lazy Loading) avec un client Redis (`ioredis 5.10.1`) encapsulé dans le module `src/lib/cache.ts`. Chaque opération dispose d’un TTL configuré : 5 minutes pour la liste des produits, 30 minutes pour les catégories, 10 minutes pour le détail d’un produit, et 3 minutes pour la liste des publications. L’invalidation est gérée en écriture (Write-Through) : toute mutation (création, vote, commentaire) supprime les clés affectées du cache.

Résultats mesurés. Le tableau 1 présente les temps de réponse en millisecondes pour trois opérations critiques. L’opération “Get Launch Detail” n’a pas pu être mesurée car la base de données ne contenait aucun produit lors du benchmark.

TABLE 1 – Résultats du benchmark Cache Redis — SaaS Directory (10 mai 2026)

Opération	Sans Cache (ms)	Cache Warm (ms)	Gain
Liste des produits	9.17	0.05	↓99.4%
Catégories	1.69	0.07	↓95.6%
Liste des publications	1.88	0.06	↓97.0%

La figure 10 illustre la comparaison visuelle des temps de réponse. Le cache warm (données en

mémoire Redis) réduit les temps de réponse de plus de 95%, passant de quelques millisecondes de requêtes SQL complexes à des réponses inférieures à 0.1 ms.

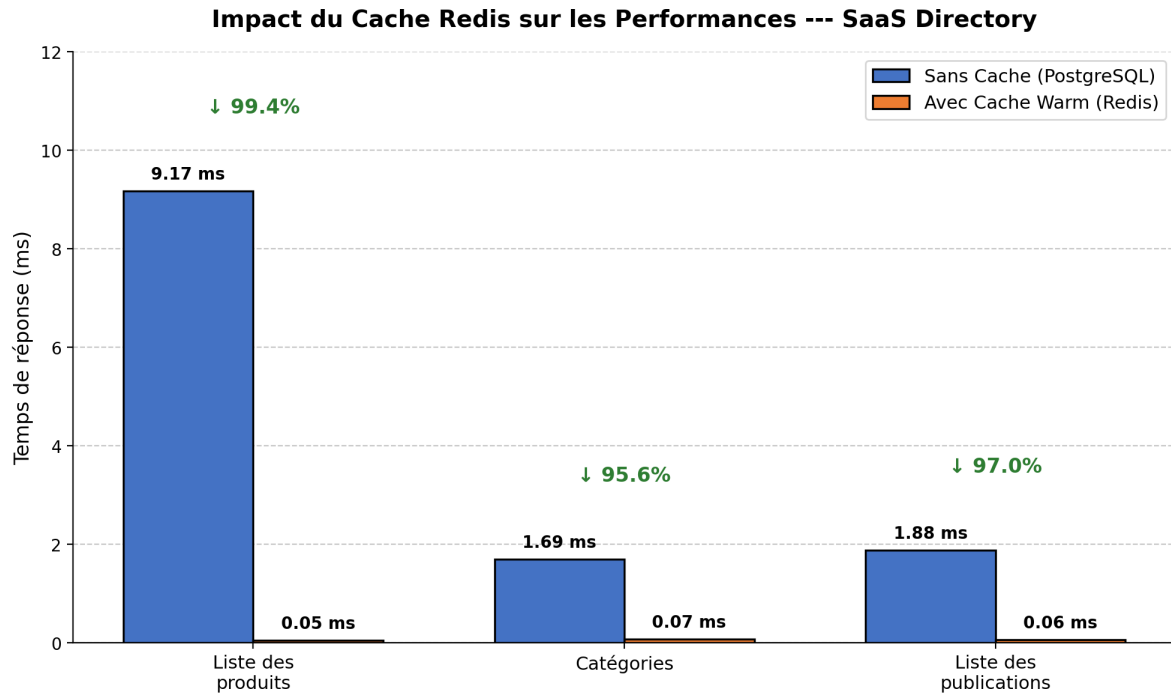


FIGURE 10 – Impact du Cache Redis sur les Performances — Comparaison avant/après

6.2 Outils et Environnement de Test

L'environnement de test utilise une base PostgreSQL dédiée sur le port 5433. La séparation des environnements (développement, test, production) est assurée par des variables d'environnement dédiées.

6.3 Couverture et Résultats

La couverture de tests cible les fonctions critiques : authentification (création de compte, connexion, 2FA), gestion des rôles, marketplace (création de produit, vote, commentaire), messagerie (envoi, réception), et abonnements (souscription, changement de plan). Les seuils de couverture sont configurés avec un minimum de 70%.

La gestion des projets logiciels vise à structurer méthodiquement une réalité à venir, en appliquant un objectif et des actions à entreprendre avec des ressources données.

7.1 Organisation du Projet (WBS, PBS, OBS)

Trois outils structurants ont guidé l'organisation du projet.

Le **Work Breakdown Structure (WBS)** décompose le projet en tâches hiérarchiques : fondations, infrastructure de tests, base de données, authentification, fonctionnalités core, et polissage.

Le **Product Breakdown Structure (PBS)** identifie les livrables : authentification fonctionnelle, profil utilisateur opérationnel, marketplace avec votes et commentaires, système de messagerie, et abonnements Free/Pro.

Le **Organizational Breakdown Structure (OBS)** répartit les responsabilités entre les trois développeurs du projet.

7.2 Planification (GANTT, PERT)

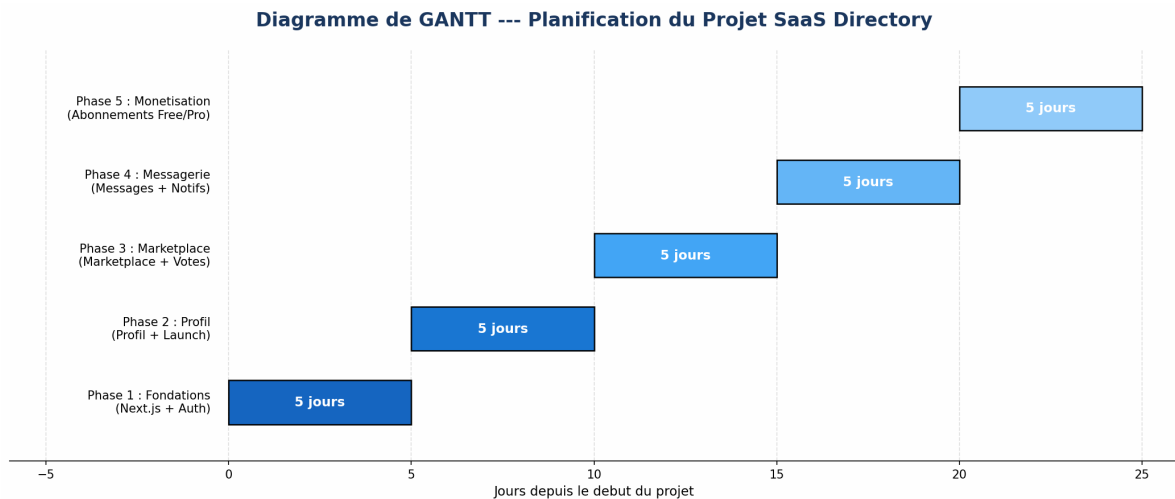


FIGURE 11 – Diagramme de GANTT — Planification du Projet SaaS Directory

La figure 11 présente une planification hypothétique à visée pédagogique sur une échelle de 130 jours, illustrant le découpage type d'un projet web de cette envergure. Le diagramme PERT modélise les dépendances entre tâches de type “fin à début”.

7.3 Estimation des Coûts par COCOMO

L'estimation des coûts d'un projet logiciel est un exercice délicat. Pour contrer les biais classiques (optimisme, négligence technique), nous avons appliqué la méthode COCOMO (CONstructive COSt MOdel) de Barry Boehm.

7.3.1 Taille du Projet

Le codebase de SaaS Directory compte 12 166 lignes de code réparties sur 108 fichiers TypeScript/TSX, soit approximativement 14 KLOC. Répartition principale : application (2 309 LOC), composants (5 234 LOC), utilitaires serveur et librairies (2 957 LOC), tests unitaires et d'intégration (1 083 LOC), tests end-to-end (33 LOC).

7.3.2 Application du COCOMO de Base

Pour un projet web moderne (Next.js 16.2.4, React 19.2.4, TypeScript 5), nous avons retenu le mode semi-détaché. Les coefficients sont $a = 3,0$ et $b = 1,12$.

TABLE 2 – Estimation COCOMO de Base — Mode Semi-Détaché (14 KLOC)

Paramètre	Formule / Valeur	Résultat
Taille du projet	12 166 LOC $\approx$ 14 KLOC	14 KLOC
Mode de développement	Semi-Détaché	$a = 3,0; b = 1,12$
Effort estimé	$E = a \times (\text{KLOC})^b$	$E \approx 58$ Personnes-Mois
Durée estimée	$D = 2,5 \times E^{0,35}$	$D \approx 9,5$ mois
Taille d'équipe théorique	$N = E/D$	$N \approx 6$ personnes
Taille d'équipe réelle	3 développeurs sur 3 semaines	2,3 Personnes-Mois

Ces chiffres constituent une estimation théorique à visée pédagogique. En réalité, le projet a été réalisé par trois développeurs sur environ 2,3 Personnes-Mois (17 jours de développement), illustrant l'écart fréquent entre les modèles prédictifs et la réalité des projets académiques intensifs.

L'analyse des risques est une composante essentielle de la conduite de projet. Elle vise à identifier, évaluer et traiter les événements incertains qui pourraient affecter négativement les objectifs.

8.1 Identification des Risques

Trois catégories principales de risques ont été identifiées.

8.1.1 Risques Techniques

Le risque technique majeur réside dans la complexité de Next.js 16.2.4 avec le App Router. Ce framework introduit des breaking changes significatifs par rapport aux versions précédentes. Un

risque secondaire concerne la configuration des relations et indexes Drizzle ORM 0.45.2 + PostgreSQL, qui nécessite une expertise SQL approfondie pour garantir les performances.

8.1.2 Risques de Sécurité

Les risques de sécurité concernent la protection des données utilisateurs et la prévention des attaques. L'injection SQL est mitigée par l'utilisation de Drizzle ORM avec des requêtes paramétrées. Les risques liés à l'authentification incluent la fuite de tokens JWT et la compromission des sessions. Better Auth 1.6.9 gère nativement plusieurs de ces menaces (expiration des tokens, rotation, rate limiting).

8.1.3 Risques de Performance

Le marketplace doit supporter un grand nombre de produits sans dégradation du temps de réponse, ce qui nécessite l'optimisation des indexes PostgreSQL et la pagination côté serveur. Le cache Redis est utilisé pour réduire la charge sur la base de données pour les données fréquemment accédées.

8.2 Analyse et Stratégies de Mitigation

8.2.1 Mitigation des Risques Techniques

La mitigation repose sur trois piliers : veille technologique (documentation officielle de Next.js consultée régulièrement), tests end-to-end Playwright 1.59.1 pour détecter les régressions, et modularité de l'architecture facilitant le remplacement d'une technologie si elle devient obsolète.

8.2.2 Mitigation des Risques de Sécurité

La sécurité est assurée par plusieurs mécanismes : Better Auth gère la sécurité des sessions (tokens JWT, expiration, rotation); les Server Actions valident les entrées utilisateur via Zod 4.4.3; et les indexes PostgreSQL accélèrent les requêtes fréquentes tout en limitant l'attaque par injection.

8.2.3 Mitigation des Risques de Performance

La performance est optimisée par : les indexes PostgreSQL définis dans le schéma Drizzle, le cache Redis pour les données fréquemment accédées (sessions, profils, listes de catégories), et la pagination côté serveur pour le marketplace et la messagerie.

8.3 Plan de Continuité

En cas d'incident majeur, plusieurs mesures sont prévues : restauration depuis les backups PostgreSQL, reconstruction du schéma via les migrations Drizzle (0000 à 0003), retour à une version stable via Git (branche principale et tags), et documentation technique facilitant l'onboarding d'un nouveau développeur.

La conduite du changement est la discipline qui vise à accompagner les transformations au sein d'un projet.

9.1 Nature des Changements

Plusieurs types de changements ont affecté le projet. Les **changements technologiques** incluent l'ajout de Framer Motion pour les animations, l'intégration de Redis pour le cache côté serveur, et l'évolution du schéma Drizzle avec les migrations successives. Les **changements fonctionnels** concernent l'ajout de fonctionnalités : système de notifications, édition et suppression des commentaires, pagination infinie pour la messagerie. Les **changements organisationnels** touchent la répartition des tâches entre les trois développeurs.

9.2 Gestion des Versions

Le contrôle de versions est assuré par Git avec une stratégie de branches structurée. La branche principale constitue la version de production stable. Les branches de fonctionnalités isolent le développement. Les pull requests avec revue de code assurent la qualité avant fusion.

Les migrations Drizzle constituent un mécanisme de gestion du changement au niveau de la persistance. Chaque migration est un fichier SQL versionné (de 0000 à 0003) qui peut être appliquée et revertée. La figure 12 présente la chronologie de l'évolution du schéma.

Cette approche garantit la cohérence du schéma entre les environnements.

9.3 Adaptation et Communication

L'architecture modulaire du projet a facilité l'adaptation aux changements. La migration 0002 a ajouté le champ `logoUrl` dans la table `Launch` et les champs `isDeleted` et `updatedAt` dans la table `Message`. La migration 0003 a ajouté les préférences de notification (JSONB) dans la table `Profile`. Chaque évolution n'a nécessité que la modification du schéma Drizzle ORM et de l'interface

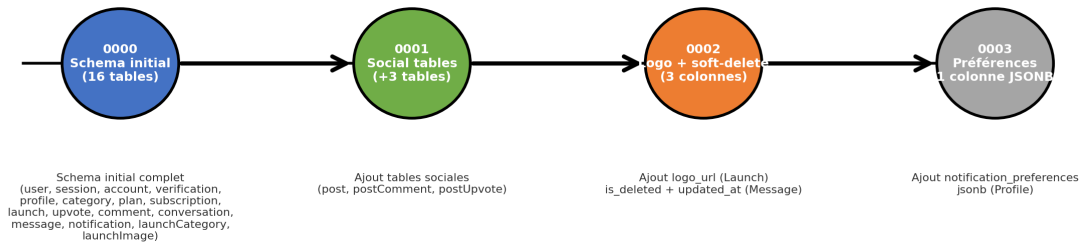


FIGURE 12 – Chronologie de l'Évolution du Schéma — Migrations Drizzle

associée.

La communication autour des changements repose sur plusieurs canaux : la documentation technique maintenue à jour, les messages de commit suivant une convention descriptive, et les discussions techniques documentées dans les revues de code.

Ce rapport a présenté l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de SaaS Directory. Du point de vue du cycle de vie, le modèle incrémental itératif adopté a permis de livrer des versions fonctionnelles successives, chacune apportant une valeur métier concrète.

La démarche orientée objet, matérialisée par le schéma Drizzle ORM de 20 tables et les composants React modulaires, assure la maintenabilité et l'évolutivité du système. Les sept diagrammes UML (cas d'utilisation, classes, séquence, activité, état, déploiement, architecture en couches) et les trois visualisations de gestion de projet (GANTT, pyramide des tests, chronologie des migrations) fournissent une documentation visuelle complète du projet.

La gestion de projet, illustrée par le WBS, le diagramme de GANTT, et l'estimation COCOMO, démontre la faisabilité d'une planification rigoureuse même dans un contexte académique. L'analyse des risques a permis d'anticiper et de mitiger les principales menaces techniques et sécuritaires.

L'utilisation d'une stack technologique moderne et cohérente — Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, Better Auth 1.6.9, PostgreSQL, et Redis — a grandement facilité le développement et garanti la qualité du livrable final.

Les perspectives d'évolution de SaaS Directory sont nombreuses. L'intégration complète de Stripe pour le paiement (le schéma contient déjà les champs client et abonnement Stripe), l'ajout d'un système de recherche avancée, et le déploiement sur une infrastructure cloud avec Docker et Kubernetes constituent les axes de développement futurs.

- Barry Boehm, *Software Engineering Economics*, Prentice-Hall, 1977.
- Grady Booch, *Object-Oriented Analysis and Design with Applications*, 2nd Edition, Benjamin/Cummings, 1994.
- Jean-Raymond Abrial, *The B-Book : Assigning Programs to Meanings*, Cambridge University Press, 1996.
- ISO/IEC 12207 :2017, Systems and software engineering — Software life cycle processes.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 1028-2008, IEEE Standard for Software Reviews and Audits.
- IEEE Std 12207.0-1996, IEEE Standard for Information Technology — Software life cycle processes.
- Cours de Génie Logiciel et Conduite de Projet, Année Universitaire 2025-2026.
- Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- React Documentation, <https://react.dev>, consulté en 2026.
- Drizzle ORM Documentation, <https://orm.drizzle.team>, consulté en 2026.
- Better Auth Documentation, <https://www.better-auth.com>, consulté en 2026.
- Tailwind CSS Documentation, <https://tailwindcss.com>, consulté en 2026.
- PostgreSQL Documentation, <https://www.postgresql.org/docs>, consulté en 2026.
- Redis Documentation, <https://redis.io/documentation>, consulté en 2026.
- Vitest Documentation, <https://vitest.dev>, consulté en 2026.
- Playwright Documentation, <https://playwright.dev>, consulté en 2026.

Annexe A : Dependances Principales

TABLE 3 – Dependances principales du projet SaaS Directory

Dependance	Version	Description
next	16.2.4	Framework React avec App Router
react	19.2.4	Bibliotheque UI
react-dom	19.2.4	Rendu DOM React
drizzle-orm	0.45.2	ORM type-safe pour PostgreSQL
better-auth	1.6.9	Authentification OAuth + 2FA
pg	8.20.0	Driver PostgreSQL
@tanstack/react-query	5.100.9	Gestion de cache et requetes
zod	4.4.3	Validation de schemas
framer-motion	12.38.0	Animations React
ioredis	5.10.1	Client Redis
resend	6.12.2	Service d'envoi d'emails
sonner	2.0.7	Notifications toast UI
tailwindcss	^4	Framework CSS utilitaire
typescript	^5	Typage statique
vitest	4.1.5	Tests unitaires et d'integration
eslint	^9	Linting du code
eslint-config-next	16.2.4	Regles ESLint Next.js

Annexe B : Diagramme de Deploiement

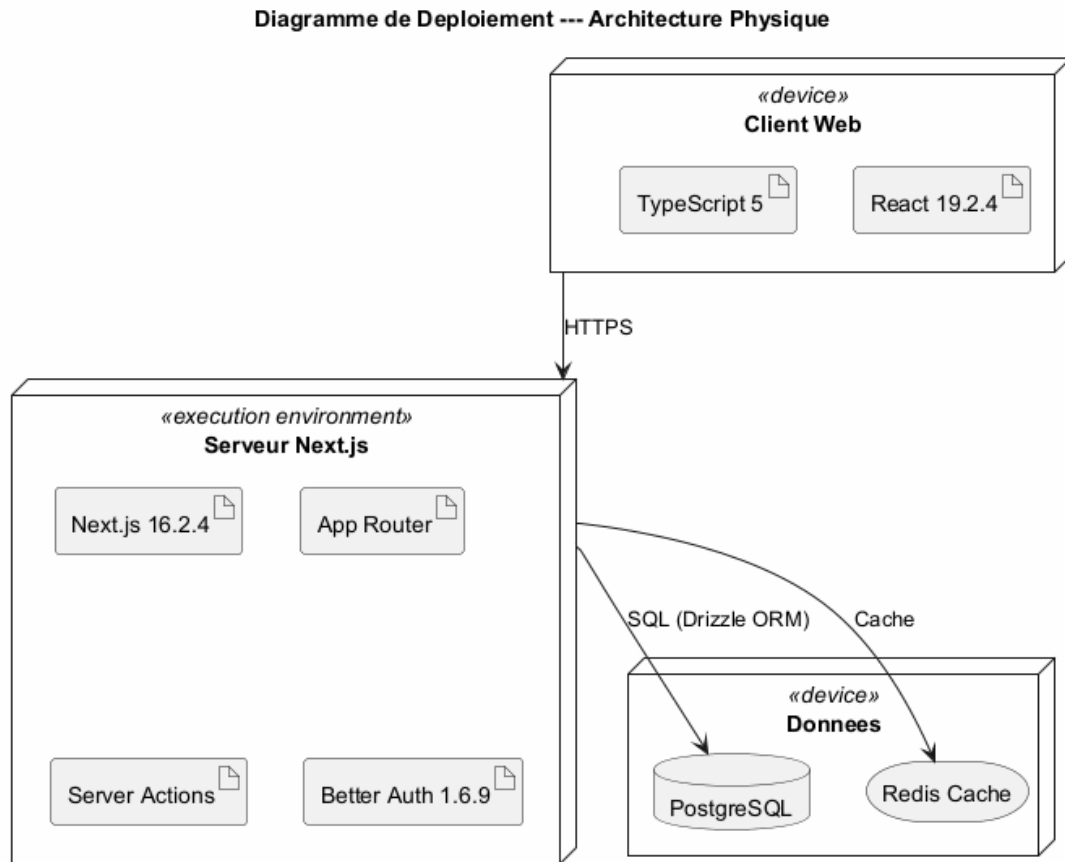


FIGURE 13 – Diagramme de Deploiement — Architecture Physique de SaaS Directory

La figure 13 décrit l'architecture physique du système. SaaS Directory repose sur une architecture distribuée sur trois environnements principaux : le client web (navigateur), la plateforme serveur (Next.js 16.2.4), et les services de données (PostgreSQL + Redis).

Le client web (React 19.2.4 / TypeScript 5) s'exécute dans le navigateur et communique avec les fonctions serverless via HTTPS. Les Server Actions Next.js 16.2.4 interagissent avec la base de données PostgreSQL via Drizzle ORM 0.45.2. Redis est utilisé comme cache côté serveur pour réduire la charge sur la base de données. L'authentification est assurée par Better Auth 1.6.9.